Software Tools and Process Automation

Introduction, Version Control, CI/CD, DevOps, Review, Final Quiz, and References.

Quiz: 10 multiple-choice questions (easy to hardest), followed by 5 easy recall questions. No answer key is included.

What is Software Tools and Process Automation?

- Software tools help people create, test, manage, and operate software.
- Process automation uses defined rules and tools to perform repeatable steps with less manual effort.
- Together, they support a repeatable path from an idea to a running application.

What is Software Tools and Process Automation?

Examples include Git, CI/CD workflows, and deployment practices. Tools are helpers; automation implements repeatable processes.

Why does it matter?

- Repetitive manual steps take time and can be performed inconsistently.
- Automated builds and tests provide earlier feedback; version history makes changes traceable.
- Repeatable processes help teams collaborate, but still need human judgment, maintenance, and security.

Why does it matter?

Compare manually copying project folders with reviewing changes in Git and running the same checks.

Automation does not guarantee bug-free software.

Who uses these tools and processes?

- Developers and students track code and collaborate using version control.
- QA and test engineers automate checks and investigate quality problems.
- Operations, platform, and security teams automate environments, delivery, and monitoring.

Who uses these tools and processes?

Responsibilities overlap. DevOps encourages shared ownership instead of assigning all automation to one person.

One student project. A connected workflow.

- A developer changes a React task-management interface and records the change in Git.
- A CI workflow builds and tests it; the team reviews results before releasing.
- The team operates the application, monitors it, and uses feedback to improve the next change.

```
Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor

Monitoring feeds the next plan.
```

One student project. A connected workflow.

Preview the deeper topics. The final quiz draws only on material presented in the lessons.

What do software tools help people do?

A. Avoid collaboration

B. Create, test, manage, and operate software

C. Guarantee perfect software

D. Remove all planning

Which task is a good candidate for process automation?

A. Every human judgment

B. All team conversations

C. A repeatable build-and-test step

D. Choosing every product goal

Why do teams automate builds and tests?

A. To get earlier, repeatable feedback

B. To guarantee no bugs exist

C. To eliminate security work

D. To avoid reviewing changes

Who uses software tools and automation?

A. Only game designers

B. Only operations teams

C. Only professional programmers

D. Students, developers, test engineers, and operations teams

Every great adventure has a history.

- Understand why teams track changes.
- Read a Git workflow and branch graph.
- Distinguish Git from GitHub and GitLab.

Every great adventure has a history.

Ask: how would you recover yesterday's working version? Introduce the three objectives.

Save the story of your code.

- Version control records changes to files over time.
- Compare versions, recover earlier work, and understand who changed what.
- A shared history helps teams collaborate.

Save the story of your code.

A commit is a meaningful snapshot, not just a backup. Version control can track more than source code.

One central copy? Or many?

- Centralized VCS: clients depend on a central version database.
- Distributed VCS: clones include repository history.
- Git is distributed; most local operations work offline.

One central copy? Or many?

A remote is useful for collaboration, but Git does not require GitHub. A clone is not necessarily a copy of every remote branch checkout.

Meet Git, your history engine.

- Repository: files and their recorded history. Commit: a recorded snapshot.
- Branch: a movable line of development. Merge: integrate histories.
- Clone copies a repository; push sends commits; pull fetches and integrates.

```
main: o ---- o -------- o

feature:       \ -- o -- /
```

Meet Git, your history engine.

Pull integrates using merge or rebase depending on configuration. Branches let work evolve independently.

Four steps. A repeatable rhythm.

- Modify your working files.
- Stage the changes you intend to include.
- Commit locally, then push to a shared remote.

```
Modify -> Stage -> Commit -> Push
```

Four steps. A repeatable rhythm.

The staging area lets you choose the next snapshot. Push is separate from commit.

GitHub: collaboration around Git.

- Host repositories and discuss work through issues.
- Propose changes with pull requests and code review.
- GitHub Actions can automate builds, tests, and other workflows.

GitHub: collaboration around Git.

A pull request is a platform collaboration feature, not a core Git command.

GitLab: a connected development platform.

- Host Git repositories and organize collaboration.
- Use merge requests to review and integrate changes.
- Define GitLab CI/CD jobs in .gitlab-ci.yml.

GitLab: a connected development platform.

GitLab pipelines group jobs into stages; runners execute jobs. A pipeline need not deploy.

The engine and the platforms.

- Git -> distributed version control software.
- GitHub -> Git hosting, pull requests, issues, and Actions.
- GitLab -> Git hosting, merge requests, and integrated CI/CD.

The engine and the platforms.

Both platforms use Git underneath. Teams choose based on workflow, hosting, and organizational needs.

Ship a React login feature.

- Clone an existing repository first.

- Create a feature branch, edit React components, and run checks.

- Commit and push, then open a pull or merge request.

```
git clone https://example.com/team/student-tasks.git
cd student-tasks
git switch -c feature-login
git add .
git commit -m "Add login feature"
git push -u origin feature-login
```

Ship a React login feature.

Replace the example URL with your actual remote. These commands assume a configured Git identity and existing repository.

Checkpoint: your history is safe.

- Git tracks changes; platforms help teams collaborate.
- Small, clear commits make review easier.
- Use branches and review before integrating work.

Checkpoint: your history is safe.

Recap modify, stage, commit, push. Next is the Version Control Bowser phase; the knowledge quiz is the final map stop.

What does version control record?

A. Only passwords

B. Only running processes

C. Changes to files over time

D. Only production errors

Which command stages changes?

A. git add

B. git push

C. git clone

D. git pull

What is a Git branch useful for?

A. Installing React

B. Independent development before integration

C. Replacing every test

D. Hosting a database

Which statement about the platforms is accurate?

A. Git is a GitHub account

B. GitLab is a programming language

C. GitHub and GitLab replace Git history

D. GitHub uses pull requests; GitLab uses merge requests

Turn repetition into a pipeline.

- Explain continuous integration.
- Compare continuous delivery and continuous deployment.
- Trace a change through an automated pipeline.

Turn repetition into a pipeline.

Connect this world to the commits and pull requests from World 1.

CI/CD connects change to feedback.

- CI integrates changes frequently with automated checks.
- Delivery keeps validated changes ready for production release.
- Deployment automatically releases eligible validated changes.

CI/CD connects change to feedback.

CI alone does not imply production deployment. CD can mean delivery or deployment; always clarify.

Integrate early. Find problems sooner.

- Developers integrate small changes frequently.
- An automated workflow builds and tests integrated changes.
- Failures give feedback so the team can fix problems promptly.

```
Source -> Build -> Test -> Release -> Deploy -> Monitor

Required test failure blocks downstream release and deployment.
```

Integrate early. Find problems sooner.

CI is a practice supported by automation, not simply installing a server.

Build it. Test it. Integrate it.

- Install reproducible dependencies and compile or bundle.
- Run appropriate unit, integration, and other checks.
- Review results and fix failures before proceeding.

Build it. Test it. Integrate it.

A green build only tells us that configured checks passed. It does not prove absence of bugs.

Continuous delivery: ready to release.

- Automate validation and prepare deployable artifacts.
- Keep software in a releasable condition.
- Production release remains a deliberate approval or business decision.

Continuous delivery: ready to release.

Delivery may deploy automatically to staging while retaining a production approval.

Continuous deployment: release automatically.

- Eligible changes pass the required automated checks.
- The pipeline deploys them to production without a manual approval step.
- Monitoring and rollback strategies remain essential.

Continuous deployment: release automatically.

Automatic deployment needs reliable controls. It does not mean skipping testing.

One important gate.

- Delivery -> validated artifact -> production approval -> deploy.
- Deployment -> validated artifact -> automatic production deploy.
- Both rely on automated validation and release readiness.

One important gate.

Use the presence of a manual production approval to explain the distinction.

Follow a package through the factory.

- Source -> build -> test -> release -> deploy -> monitor.
- A failed required test stops downstream release and deployment.
- This example includes deployment; many CI workflows stop after testing.

```
Source -> Build -> Test -> Release -> Deploy -> Monitor

Required test failure blocks downstream release and deployment.
```

Follow a package through the factory.

Toggle the test result in the diagram. Release means preparing a versioned artifact here; production deployment is a separate step.

Choose tools for the workflow.

- GitHub Actions: repository workflows defined in YAML.
- GitLab CI/CD: pipelines and jobs executed by runners.
- Jenkins: extensible automation server for build and delivery workflows.

Choose tools for the workflow.

Tools enable practices. They do not replace team ownership or careful pipeline design.

A small React CI workflow.

- Run on pushes and pull requests.

- Install from the lockfile, test once, and build.

- This workflow validates code; it does not deploy.

```
name: React CI
on: [push, pull_request]
jobs:
  validate:
    runs-on: ubuntu-latest
    permissions:
      contents: read
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm test -- --run
      - run: npm run build
```

A small React CI workflow.

Assumes a committed package-lock.json and a Vitest test script. Pin actions to reviewed commit SHAs in hardened production workflows.

Fast feedback, with guardrails.

- Repeatable checks reduce manual repetition and expose problems earlier.
- Protect secrets, limit token permissions, and review dependencies.
- Flaky tests, unsafe scripts, and weak coverage limit confidence.

Fast feedback, with guardrails.

Never expose production secrets to untrusted contributions. Automation cannot eliminate security risks.

Checkpoint: automate with intent.

- CI builds confidence through frequent integration and checks.
- Delivery keeps releases ready; deployment releases automatically.
- Failed checks should stop unsafe downstream work.

Checkpoint: automate with intent.

Ask students to identify where the manual gate belongs before the CI/CD Bowser phase.

What does continuous integration commonly automate?

A. Every line of code

B. Builds and tests

C. All product decisions

D. Internet billing

What distinguishes continuous delivery from continuous deployment?

A. Delivery forbids testing

B. Deployment requires no monitoring

C. Delivery retains a deliberate production approval or release decision

D. Delivery cannot prepare artifacts

A required test fails in the illustrated pipeline. What happens?

A. Downstream release and deployment stop

B. Production always proceeds

C. The test is deleted

D. Monitoring replaces the failed test

What does a passing CI build tell us?

A. The application has no bugs

B. Security risks are eliminated

C. No review is needed

D. The configured checks passed

The final world is a team effort.

- Define DevOps as culture and practices.
- Connect development, operations, and feedback.
- See how Git and CI/CD support the complete lifecycle.

The final world is a team effort.

DevOps is not a job title or a single product. It is a way to improve collaboration and delivery.

DevOps starts with shared ownership.

- Development and operations collaborate to deliver and run software.
- Culture, practices, and tools support faster, more reliable work.
- Automation supports people; it does not replace responsibility.

DevOps starts with shared ownership.

Use AWS's definition as the starting point. Avoid reducing DevOps to an automation tool.

Build and run belong together.

- Development creates and improves application behavior.
- Operations supports availability, environments, and reliable running systems.
- Shared goals and feedback reduce handoff friction.

Build and run belong together.

Responsibilities vary by team. Avoid portraying these as rigid or opposing roles.

Culture makes the tools work.

- Collaborate across disciplines.
- Share responsibility for quality, security, and production outcomes.
- Automate repeatable tasks and learn from incidents.

Culture makes the tools work.

Blameless learning focuses on system improvement rather than hiding mistakes.

A lifecycle, not a finish line.

- Plan -> code -> build -> test.
- Release -> deploy -> operate -> monitor.
- Monitoring feeds the next plan. Security spans every phase.

```
Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor

Monitoring feeds the next plan.
```

A lifecycle, not a finish line.

This eight-phase diagram is a teaching model, not a mandated universal standard.

Describe infrastructure as code.

- Machine-readable configuration defines infrastructure.
- Version and review changes; use repeatable provisioning.
- Configuration management helps maintain desired system settings.

Describe infrastructure as code.

Examples include Terraform for provisioning and Ansible for configuration. Review plans and protect state and secrets.

Package consistently. Coordinate deployment.

- Docker packages applications and dependencies in container images.
- Kubernetes orchestrates containerized workloads.
- Automation can roll out updates, check health, and support recovery.

Package consistently. Coordinate deployment.

Containers share the host kernel. Kubernetes is useful in some environments, but is not required for DevOps.

Listen to the running system.

- Metrics show trends; logs record events; traces connect requests.
- Alerts should help people respond to actionable problems.
- Integrate security checks and operational feedback throughout delivery.

Listen to the running system.

Monitoring gives evidence, not a guarantee of reliability. Protect sensitive information in logs.

Replace handoffs with feedback.

- Silos: large batches, separate goals, and late operational feedback.
- DevOps: small changes, shared goals, and ongoing collaboration.
- Practices must fit the team and its constraints.

Replace handoffs with feedback.

This is an illustrative comparison, not a claim that every traditional team works identically.

Measure delivery to improve it.

- Change lead time: commit to production; deployment frequency: how often production deployments occur.

- Failed deployment recovery time: time to recover from a failed production deployment.

- Change fail rate: proportion requiring immediate intervention; deployment rework rate: unplanned deployments caused by production incidents.

Measure delivery to improve it.

Current DORA guidance uses five metrics. Historical "four keys" and broad MTTR labels differ. Measure a service over time, not individual developer output; avoid league tables.

Three ideas. One connected system.

- Git records the change and enables review.
- CI/CD validates and supports repeatable delivery.
- DevOps connects people, practices, operation, and learning.

```
Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor

Monitoring feeds the next plan.
```

Three ideas. One connected system.

These are complementary, not interchangeable.

Example: Student Task Management.

- React interface -> GitHub repository and pull request.
- GitHub Actions validates tests/build; an approved release deploys to Vercel.
- Supabase provides application data; the team monitors errors and access policies.

Example: Student Task Management.

Illustrative architecture only: this presentation does not connect to external services. Keep credentials out of source, review database access rules, and plan migrations.

Start small. Improve continuously.

- Challenges: siloed ownership, flaky tests, skill gaps, and legacy systems.
- Use small changes, code review, reproducible builds, and safe rollback.
- Choose useful measures and improve one bottleneck at a time.

Start small. Improve continuously.

Do not promise a universal performance uplift. Context and sustained teamwork matter.

Checkpoint: connect the whole adventure.

- Version control gives changes a history.
- CI/CD creates repeatable validation and release paths.
- DevOps turns feedback into shared improvement.

Checkpoint: connect the whole adventure.

Defeat the fourth Bowser phase to rescue the princess, then enter the final map knowledge quiz.

DevOps is best described as:

A. A single automation tool

B. A programming language

C. A culture and collection of collaborative practices

D. A replacement for operations

What is Infrastructure as Code?

A. Defining infrastructure using machine-readable configuration

B. Drawing server pictures

C. Writing only HTML

D. Manually restarting machines

What does monitoring give a team?

A. A guarantee against bugs

B. Operational feedback

C. A replacement for security

D. Source code compilation

How do Git, CI/CD, and DevOps fit together?

A. They do exactly the same thing

B. They are competing languages

C. They are only useful for games

D. Git tracks changes, CI/CD validates and delivers, and DevOps connects people and practices

Save the story of your code.

- Version control records changes to files over time.
- Compare versions, recover earlier work, and understand who changed what.
- A shared history helps teams collaborate.

Save the story of your code.

A commit is a meaningful snapshot, not just a backup. Version control can track more than source code.

The engine and the platforms.

- Git -> distributed version control software.
- GitHub -> Git hosting, pull requests, issues, and Actions.
- GitLab -> Git hosting, merge requests, and integrated CI/CD.

The engine and the platforms.

Both platforms use Git underneath. Teams choose based on workflow, hosting, and organizational needs.

Four steps. A repeatable rhythm.

- Modify your working files.
- Stage the changes you intend to include.
- Commit locally, then push to a shared remote.

```
Modify -> Stage -> Commit -> Push
```

Four steps. A repeatable rhythm.

The staging area lets you choose the next snapshot. Push is separate from commit.

CI/CD connects change to feedback.

- CI integrates changes frequently with automated checks.
- Delivery keeps validated changes ready for production release.
- Deployment automatically releases eligible validated changes.

CI/CD connects change to feedback.

CI alone does not imply production deployment. CD can mean delivery or deployment; always clarify.

Integration, delivery, deployment.

- Integration: merge frequently and validate automatically.
- Delivery: remain ready for an approved production release.
- Deployment: release eligible validated changes automatically.

Integration, delivery, deployment.

Ask the audience where approval happens.

Follow a package through the factory.

- Source -> build -> test -> release -> deploy -> monitor.
- A failed required test stops downstream release and deployment.
- This example includes deployment; many CI workflows stop after testing.

```
Source -> Build -> Test -> Release -> Deploy -> Monitor

Required test failure blocks downstream release and deployment.
```

Follow a package through the factory.

Toggle the test result in the diagram. Release means preparing a versioned artifact here; production deployment is a separate step.

DevOps starts with shared ownership.

- Development and operations collaborate to deliver and run software.
- Culture, practices, and tools support faster, more reliable work.
- Automation supports people; it does not replace responsibility.

DevOps starts with shared ownership.

Use AWS's definition as the starting point. Avoid reducing DevOps to an automation tool.

A lifecycle, not a finish line.

- Plan -> code -> build -> test.
- Release -> deploy -> operate -> monitor.
- Monitoring feeds the next plan. Security spans every phase.

```
Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor

Monitoring feeds the next plan.
```

A lifecycle, not a finish line.

This eight-phase diagram is a teaching model, not a mandated universal standard.

Three ideas. One connected system.

- Git records the change and enables review.
- CI/CD validates and supports repeatable delivery.
- DevOps connects people, practices, operation, and learning.

```
Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor

Monitoring feeds the next plan.
```

Three ideas. One connected system.

These are complementary, not interchangeable.

Your next adventure starts with one change.

- Track it with Git. Validate it with CI/CD.
- Collaborate, operate, and improve with DevOps.
- Open References for the official reading list.

Your next adventure starts with one change.

Invite questions and connect the three worlds to a class project.

What is the purpose of process automation?

A. Remove all human judgment

B. Perform repeatable steps using defined rules and tools

C. Guarantee that software has no bugs

D. Replace every software team

Which tool is a distributed version control system?

A. Docker

B. Kubernetes

C. GitHub Actions

D. Git

Which command stages changes for the next commit?

A. git add

B. git push

C. git clone

D. git pull

Which statement correctly distinguishes Git from GitHub?

A. GitHub replaces the need for version history

B. Git and GitHub are the same application

C. Git tracks versions; GitHub hosts repositories and supports pull requests

D. Git only works while connected to GitHub

A required automated test fails. In the illustrated pipeline, what should happen next?

A. Release proceeds because the code compiled

B. Downstream release and deployment stop until the failure is resolved

C. The pipeline silently deletes the test

D. Production deployment always happens first

What distinguishes continuous deployment from continuous delivery?

A. Deployment does not use automated checks

B. Delivery never prepares an artifact

C. Deployment prohibits code review

D. Eligible validated changes enter production without a manual approval gate

A pull-request workflow installs dependencies, runs tests, and builds React. It has no deployment step. What is it doing?

A. Continuous integration validation without deployment

B. Continuous deployment to production

C. Infrastructure provisioning only

D. Guaranteeing the absence of bugs

Which team behavior best demonstrates DevOps?

A. Operations receives code only after a large final handoff

B. Developers stop caring after a successful build

C. Development and operations share responsibility and use production feedback to improve work

D. All manual decisions are removed

An incident causes an unplanned production deployment to repair the system. Which DORA metric tracks the proportion of such unplanned deployments?

A. Deployment frequency

B. Change lead time

C. Failed deployment recovery time

D. Deployment rework rate

Which plan best connects the three main topics for a student application?

A. Copy folders manually, skip checks, and leave operation to someone else

B. Track changes with Git, validate and release through CI/CD, and share operational feedback through DevOps practices

C. Use GitHub as a replacement for testing and monitoring

D. Adopt a tool and assume that team collaboration is solved

What does CI stand for?

Answer: _____

Name the Git command used to stage changes.

Answer: _____

Which Git hosting platform in the presentation uses merge requests?

Answer: _____

DevOps connects Development with which other area?

Answer: _____

What practice observes running systems and provides operational feedback?

Answer: _____

References

Git / About version control

https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Git / Branching

https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

GitHub / Pull requests

https://docs.github.com/en/pull-requests

GitHub / Continuous integration

https://docs.github.com/en/actions/get-started/continuous-integration

References

GitLab / Merge requests

https://docs.gitlab.com/user/project/merge_requests/

GitLab / CI/CD pipelines

https://docs.gitlab.com/ci/pipelines/

AWS / Continuous delivery

https://aws.amazon.com/devops/continuous-delivery/

AWS / What is DevOps?

https://aws.amazon.com/devops/what-is-devops/

References

DORA / Five delivery performance metrics

https://dora.dev/guides/dora-metrics/

Docker / Containers

https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

Kubernetes / Overview

https://kubernetes.io/docs/concepts/overview/

Jenkins / Pipeline

https://www.jenkins.io/doc/book/pipeline/